

FleetOps AI

Test Report

29 / 29 test cases passed

15th China International College Students' Innovation & Entrepreneurship Competition
Track: AI + Construction Machinery → AI + Operation

1. Testing Objectives

This report documents the validation testing performed on FleetOps AI before submission. Testing focuses on three questions: (1) Are the underlying calculations correct? (2) Do the AI agent's insights match the known, deliberately embedded issues in the dataset? and (3) Does the system handle unexpected or malformed input without failing?

2. Test Methodology

Because the project dataset is synthetic, it was deliberately generated with known "ground-truth" issues (e.g. three units seeded with abnormal downtime, one unit seeded with a mid-period fuel-consumption spike). This allows every detection function to be tested against a known correct answer, not just checked for "does it run." All tests below were executed programmatically via `src/test_suite.py` and results were captured automatically — not manually transcribed.

Category	What it checks
Data integrity	Verify the generated dataset has no corruption, missing values, or unexpected shape
Calculation accuracy	Cross-check tool-computed KPIs against an independent manual calculation
Detection accuracy	Confirm tools correctly identify the known, deliberately injected issues
Fairness / bug regression	Confirm a previously found bug (unfair cross-type fuel comparison) stays fixed
Agent routing	Confirm each of the project's five core example questions reaches the correct analysis function
Robustness	Confirm empty, nonsensical, or malformed input does not crash the system
Build integrity	Confirm the PDF report and PPTX presentation are generated and non-empty

3. Results Summary

Metric	Value
Total test cases	29
Passed	29 (100%)
Failed	0
Test execution	Automated (src/test_suite.py)

4. Detailed Test Cases

ID	Test	Expected	Actual	Result
TC-01	Data integrity (row count, no nulls, 30 units)	2700 rows, 30 units, 0 null values	2700 rows, 30 units, 0 nulls	PASS
TC-02	KPI calculation accuracy (manual vs tool, EQ-001)	Utilization match within 0.5% (manual=96.7%)	Tool output=96.7%	PASS
TC-03	Fair fuel comparison (within-type, not fleet-wide)	Not all Dump Trucks flagged just for being naturally fuel-heavy	0/8 Dump Trucks flagged for fuel	PASS
TC-04	Fuel anomaly detection (ground truth: EQ-008 was injected with a fuel spike)	EQ-008 appears in anomalies list	Detected units: ['EQ-008']	PASS
TC-05	ML anomaly detection cross-validates manual downtime ranking	At least 2 of top-3 manual downtime units also flagged by Isolation Forest	Overlap: {'EQ-013', 'EQ-026', 'EQ-024'} (3/3)	PASS
TC-06	Agent routing: "Which equipment has the highest downtime?"	Routes to get_top_downtime_equipment	Routed to get_top_downtime_equipment	PASS
TC-06	Agent routing: "Which equipment needs urgent attention?"	Routes to get_equipment_needing_attention	Routed to get_equipment_needing_attention	PASS
TC-06	Agent routing: "Why did utilization drop?"	Routes to explain_utilization_trend	Routed to explain_utilization_trend	PASS
TC-06	Agent routing: "What's causing the fuel increase?"	Routes to analyze_fuel_consumption	Routed to analyze_fuel_consumption	PASS
TC-06	Agent routing: "Generate a fleet performance report"	Routes to generate_fleet_report	Routed to generate_fleet_report	PASS
TC-07	Edge case handling: (empty input)	No crash, returns a safe fallback answer	Returned valid response, no exception	PASS
TC-07	Edge case handling: (vague question, Arabic)	No crash, returns a safe fallback answer	Returned valid response, no exception	PASS
TC-07	Edge case handling: (random gibberish)	No crash, returns a safe fallback answer	Returned valid response, no exception	PASS

ID	Test	Expected	Actual	Result
TC-07	Edge case handling: (numeric-only input)	No crash, returns a safe fallback answer	Returned valid response, no exception	PASS
TC-08a	Project report PDF generated successfully	File exists, size > 1KB	Exists=True	PASS
TC-08b	Presentation PPTX generated successfully	File exists, size > 1KB	Exists=True	PASS
TC-09a	Top-N respected for downtime	Exactly 3 units	3 units	PASS
TC-09b	Top-N + actions for maintenance attention	Exactly 3 units with recommended actions	3 units	PASS
TC-09c	Top-N + explanations for Attention Score	Exactly 3 units with reason/action	3 units	PASS
TC-10	Project-specific utilization filtering	Scope=Makkah Housing	Scope=Makkah Housing	PASS
TC-11	Invalid equipment ID validation	Clear equipment_not_found response	equipment_not_found	PASS
TC-12	Fleet report management actions	Exactly 3 management actions	3 actions	PASS
TC-13	Arabic routing + Arabic actionable answer	3 units, Arabic reason/action	Tool=get_equipment_needing_attention, units=3	PASS
TC-14	7-day Maintenance Planner	6 prioritized units with valid schedule/action/service estimate	units=6, service_h=23.7	PASS
TC-15	What-If maintenance delay simulator	Valid 7-day scenario with impact + recommendation	extra_down=1.17, impact=High	PASS
TC-16	What-If invalid equipment validation	equipment_not_found response	equipment_not_found	PASS
TC-17a	Agent routing: Maintenance Planner	build_maintenance_plan top_n=5 horizon=7	build_maintenance_plan {'top_n': 5, 'horizon_days': 7}	PASS
TC-17b	Agent routing: What-If Simulator	simulate_maintenance_delay EQ-024 delay=7	simulate_maintenance_delay {'delay_days': 7, 'equipment_id': 'EQ-024'}	PASS
TC-18	Agent structured response for cards	tool + structured result + answer	tool=compute_attention_scores	PASS

5. Notable Finding During Testing

Issue found: the first version of `get_equipment_needing_attention()` compared every unit's fuel efficiency against the fleet-wide average. Because Dump Trucks are inherently more fuel-hungry than smaller equipment (larger engine), this flagged most Dump Trucks as "problems" even when they were operating completely normally for their type — a false positive that would have misdirected a fleet manager's attention.

Fix: the comparison was changed to measure each unit's fuel efficiency against the average of *its own equipment type* rather than the whole fleet. TC-03 was added specifically to make sure this fix stays in place if the code changes in the future.

6. Known Limitations

- Testing was performed on synthetic data with known, injected issues. Real telematics data has not yet been tested; thresholds (e.g. the 15% downtime cutoff) may need recalibration once real operational data is available.
- The natural-language agent currently ships with two modes: a keyword-based router (used for all tests above, since it requires no API key) and a real OpenAI function-calling mode. The real-LLM path was validated for correct code structure but not load-tested against a live API key in this environment due to network restrictions in the development sandbox.
- Isolation Forest contamination (currently 15%) is a fixed assumption; it has not been tuned against a labeled real-world dataset.

7. Conclusion

All 29 automated test cases passed. Core calculations were verified against independent manual computation, detection logic was verified against deliberately known ground-truth issues, and the system was confirmed to handle malformed input safely. The prototype is considered validated for demonstration and submission.